



IBM Developer  
SKILLS NETWORK

# Winning Space Race with Data Science

Swaroop Rao  
01/09/2025



# Outline

---

- Executive Summary
- Introduction
- Methodology
- Results
- Conclusion
- Appendix

# Executive Summary

---

- Methodologies used in this study were data collection, data wrangling, data cleansing, data validation, exploratory data analysis (EDA) and machine learning
- Based on the results of the analysis, we believe that we have identified factors that impact the probability of successful landings for the stage 1 booster. This equips us with the knowledge to make forward-looking decisions that will create revenue-generation opportunities in the world of space flight.

# Introduction

---

- The objective of this study is to estimate the probability of a successful landing for the first stage booster of a SpaceX rocket, given its dependencies on various factors like launch location, payload mass, orbit and others
- Problems that we are hoping to find answers to as a result of this study are:
  - Which launch site has the highest %age of success?
  - What payload mass results in the highest %age of successful launches?
  - What payload mass adversely impacts the %age of successful launches?
  - What type of booster engine has resulted in the highest %age of successful launches?



Section 1

# Methodology

# Methodology

---

## Executive Summary

- Data collection methodology:
  - Data was collected from two main sources:
    - The SpaceX API end point (<http://api.spacexdata.com/v4/>) was used to collect data about SpaceX launches
    - The Wikipedia page for SpaceX launches has useful information that supplements the data from the SpaceX API end point
- Performed data wrangling
  - There are various (8) types of landing outcomes but for the purpose of this study, we only care about successful and unsuccessful launches, so we performed data wrangling by mapping the 8 outcomes to a Boolean value of 0 (failure) and 1 (success).
- Performed exploratory data analysis (EDA) using visualization and SQL
- Performed interactive visual analytics using Folium and Plotly Dash
- Performed predictive analysis using classification models
  - We used GridSearchCV to identify the best parameters for multiple classification models like Logistic Regression, Decision Tree, Support Vector Machine (SVM) and K-Nearest Neighbor (KNN)

# Data Collection

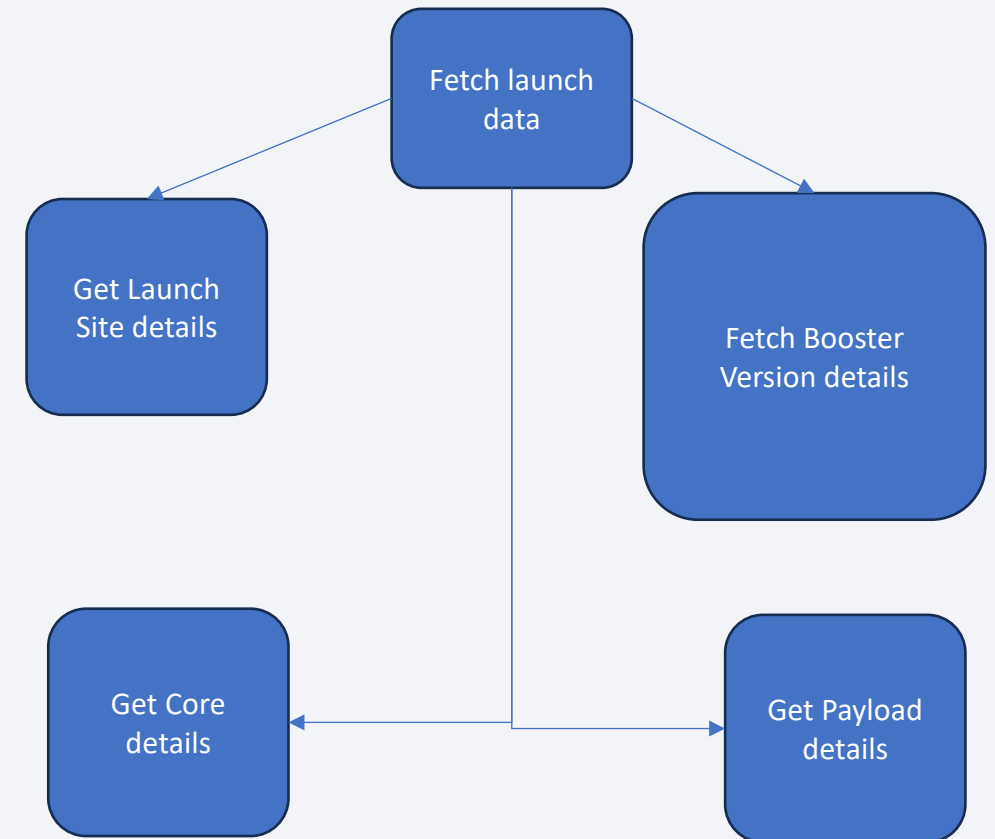
---

- Data sets were collected from two primary sources:
  - SpaceX's data API end point - <https://api.spacexdata.com/v4/>
    - We replaced missing values for Payload Mass with the mean of the other values
    - We replaced missing values for Landing Pad with the value *None*
  - Wikipedia's page for SpaceX - [https://en.wikipedia.org/wiki/List\\_of\\_Falcon\\_9\\_and\\_Falcon\\_Heavy\\_launches](https://en.wikipedia.org/wiki/List_of_Falcon_9_and_Falcon_Heavy_launches)
    - We used the BeautifulSoup library to scrape data from the Wikipedia page
    - Specifically, we scraped data from a table containing information about SpaceX rocket launches

# Data Collection – SpaceX API

---

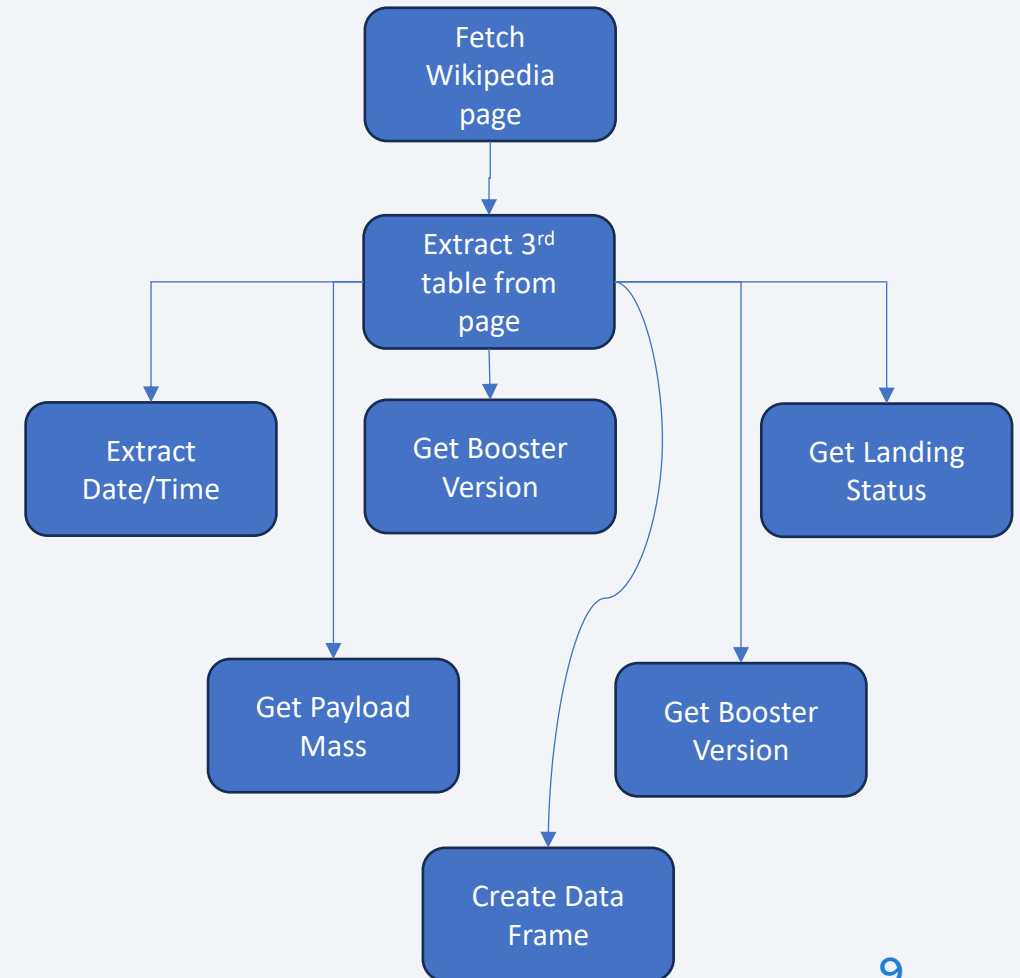
- SpaceX's data API end point - <https://api.spacexdata.com/v4/>
  - We replaced missing values for Payload Mass with the mean of the other values
  - We replaced missing values for Landing Pad with the value *None*
  - We cross-referenced the various IDs in the data using other SpaceX API end points to get more details like name, longitude, latitude, etc.
- Github URL:  
[https://github.com/swar00p/data\\_analytics/blob/c2c55be8c0e9db0303d5f99271467ef811e26d99/Capstone/jupyter-labs-spacex-data-collection-api-v2.ipynb](https://github.com/swar00p/data_analytics/blob/c2c55be8c0e9db0303d5f99271467ef811e26d99/Capstone/jupyter-labs-spacex-data-collection-api-v2.ipynb)





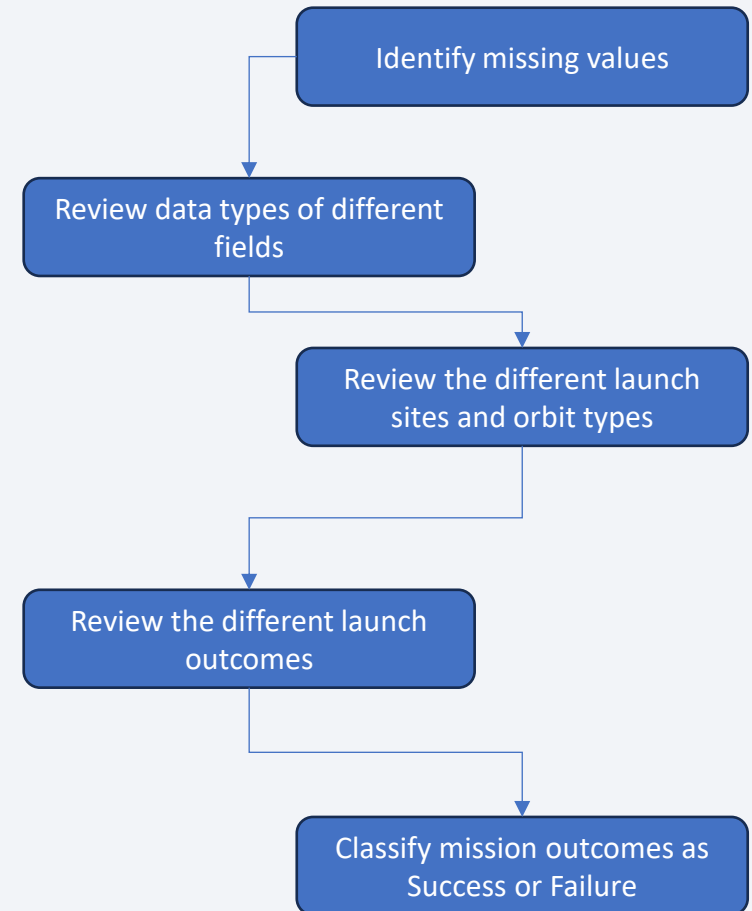
# Data Collection - Scraping

- We used the BeautifulSoup API to parse the HTML in the Wikipedia page.
- We identified the third table in the page as the one containing the details of all the launches for the Falcon booster.
- We extracted the third table from the page using the BeautifulSoup API.
- For each row in the table, we parse different columns and extract the information we are interested in like date/time of launch, launch result, orbit, customer name, etc.
- Github URL:  
[https://github.com/swar00p/data\\_analytics/blob/c2c55be8c0e9db0303d5f99271467ef811e26d99/Capstone/jupyter-labs-webscraping.ipynb](https://github.com/swar00p/data_analytics/blob/c2c55be8c0e9db0303d5f99271467ef811e26d99/Capstone/jupyter-labs-webscraping.ipynb)



# Data Wrangling

- In this step, we started by visually examining key aspects of the data
  - We reviewed the data types of the different features (columns)
  - We looked for missing data in each column to understand if it was acceptable or if steps will need to be taken to address the issue
  - We replaced missing values of Payload Mass with the average of the remaining values
  - We replaced missing values of Landing Pad with a value of *None*
  - We identified all the different types of landing results and partitioned them into Success and Failure classes.
  - We added a new column called Class that indicated if a launch was a success (1) or a failure (0).
- Github URL:  
[https://github.com/swar00p/data\\_analytics/blob/c2c55be8c0e9db0303d5f99271467ef811e26d99/Capstone/labs-jupyter-spacex-Data%20wrangling-v2.ipynb](https://github.com/swar00p/data_analytics/blob/c2c55be8c0e9db0303d5f99271467ef811e26d99/Capstone/labs-jupyter-spacex-Data%20wrangling-v2.ipynb)



# EDA with Data Visualization

---

- The following charts were plotted to understand relationships among the different variables
  - **A scatter plot of flight number vs payload mass** – as the flight number increases (i.e. for later flights), the first stage is more likely to land successfully. Also, the heavier the payload, the more likely that the first stage will not return.
  - **A scatter plot of flight number vs launch site** – as the flight number increases (i.e. for later flights), it seems that one of the launch sites (VAFB SLC-4E) stops getting used. Also, a significantly higher number of launches happened from one of the launch sites (CCAFS SLC-40). Finally, the proportion of successful launches seems to increase with increasing flight numbers.
  - **A scatter plot of payload mass vs launch site** – one of the launch sites has no launches with payload mass greater than 10000 kg. Also, there is generally a high concentration of payloads between 2000 kg and 7000 kg.
  - **Bar chart depicting success rates of different orbits for the launch** – The orbits with the highest success rate are ES-L1, GEO, HEO and SSO.
  - **A scatter plot of flight number vs orbit type** – Generally speaking, the orbits SEO, HEO, MEO, VLEO, SO and GEO appeared for later flight numbers. The earlier flight numbers were mostly for GTO, PO, LEO and ISS
  - **A scatter plot of payload mass vs orbit type** – For heavier payloads, it seems that there are more successful launches for LEO, ISS & PO orbit types.
  - **A line chart (trend) of average yearly success rate for different years** – Generally speaking, a clear upward trend was observed in success rate over the years from 2009 to 2021, except for a slight decrease in 2018.
- Github URL:  
[https://github.com/swar00p/data\\_analytics/blob/bd5425ac086d0bcbecd0ccb058e21bbab7e7a81a/Capstone/jupyter-labs-eda-dataviz-v2.ipynb](https://github.com/swar00p/data_analytics/blob/bd5425ac086d0bcbecd0ccb058e21bbab7e7a81a/Capstone/jupyter-labs-eda-dataviz-v2.ipynb)

# EDA with SQL

---

- After the preceding steps, the cleansed data was loaded into a database and analyzed using the following SQL queries:
  - `SELECT DISTINCT Launch_Site FROM SPACEXTABLE;`
  - `SELECT * FROM SPACEXTABLE WHERE Launch_Site LIKE 'CCA%' LIMIT 5;`
  - `SELECT SUM(PAYLOAD_MASS__KG_) FROM SPACEXTABLE WHERE Customer LIKE 'NASA (CRS)';`
  - `SELECT AVG(PAYLOAD_MASS__KG_) FROM SPACEXTABLE WHERE Booster_Version LIKE 'F9 v1.1'`
  - `SELECT MIN(Date) FROM SPACEXTABLE WHERE Landing_Outcome = 'Success (ground pad)';`
  - `SELECT Booster_Version FROM SPACEXTABLE WHERE Landing_Outcome = 'Success (drone ship)' AND PAYLOAD_MASS__KG_ BETWEEN 4000 AND 6000;`
  - `SELECT COUNT(*) FROM SPACEXTABLE WHERE Landing_Outcome LIKE 'Success%' OR Landing_Outcome LIKE 'Failure%';`
  - `SELECT Booster_Version FROM SPACEXTABLE WHERE PAYLOAD_MASS__KG_ = (SELECT MAX(PAYLOAD_MASS__KG_) FROM SPACEXTABLE);`
  - `SELECT SUBSTR(Date, 6, 2), Landing_Outcome, Booster_Version, Launch_Site FROM SPACEXTABLE WHERE Landing_Outcome = 'Failure (drone ship)' and SUBSTR(Date, 0, 5) = '2015';`
  - `SELECT Landing_Outcome, COUNT(*) AS NumLandings FROM SPACEXTABLE WHERE Date BETWEEN '2010-06-04' AND '2017-03-20' GROUP BY Landing_Outcome ORDER BY NumLandings DESC;`
- Github URL:  
[https://github.com/swar00p/data\\_analytics/blob/bd5425ac086d0bcbecd0ccb058e21bbab7e7a81a/Capstone/jupyter-labs-eda-sql-coursera\\_sqlite-v2.ipynb](https://github.com/swar00p/data_analytics/blob/bd5425ac086d0bcbecd0ccb058e21bbab7e7a81a/Capstone/jupyter-labs-eda-sql-coursera_sqlite-v2.ipynb)

# Build an Interactive Map with Folium

---

- An interactive map was created to allow a geospatial analysis of the data to be performed. For this, the Folium library was used which provides a toolkit of various widgets that can be manipulated with geospatial maps.
- Using Folium, we used a variety of objects like maps, circles, markers, marker clusters, lines (Polyline objects) and MousePosition to help the users visualize the data easily.
- Using these objects, the user would find it very easy to view all the launches at different launch sites on a map, visually compare success and failure launches (denoted by the marker color) and see if there are any patterns regarding the proximity of launch sites to geospatial features like the coastline, populated cities, highways and/or railway lines.
- Github URL:  
[https://github.com/swar00p/data\\_analytics/blob/bd5425ac086d0bcbecd0ccb058e21bbab7e7a81a/Capstone/lab-jupyter-launch-site-location-v2.ipynb](https://github.com/swar00p/data_analytics/blob/bd5425ac086d0bcbecd0ccb058e21bbab7e7a81a/Capstone/lab-jupyter-launch-site-location-v2.ipynb)



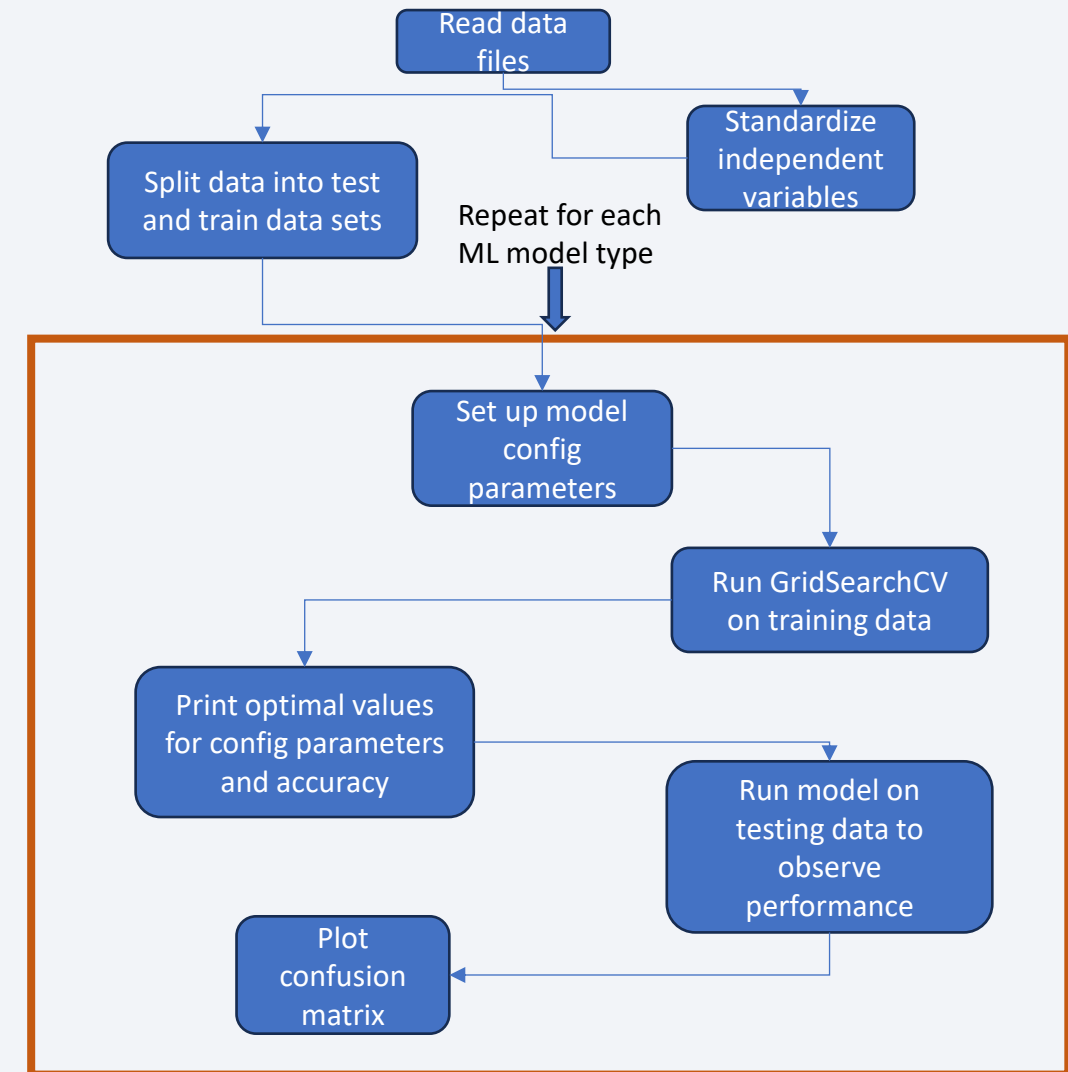
# Build a Dashboard with Plotly Dash

---

- A dashboard was created using the Plotly Dash library to give the user a convenient way to view key data insights in a single pane. The following elements were available in the dashboard:
  - A pie chart displaying the launch success rate for all launch sites or for a selected launch site
  - A scatter plot of launch results vs payload masses for all launch sites or for a selected launch sites
- A dropdown was provided to allow the user to select all launch sites or an individual launch site for both dashboard elements. Also, a slider was provided to filter the scatter plot for different ranges of the payload masses
- Using the above two dashboard widgets, the user would find it easy to make a decision regarding the ideal location for a launch site as well the payload mass that would mitigate risk of launch failures. These insights would be useful in making a strategic decision regarding the potential to launch a new competitor in the space launch industry.
- Github URL:  
[https://github.com/swar00p/data\\_analytics/blob/bd5425ac086d0bcbecd0ccb058e21bbab7e7a81a/Capstone/spacex\\_dash\\_app.py](https://github.com/swar00p/data_analytics/blob/bd5425ac086d0bcbecd0ccb058e21bbab7e7a81a/Capstone/spacex_dash_app.py)

# Predictive Analysis (Classification)

- The last step was to develop a machine learning model that would analyze the data and be able to predict the probability of launch success with a high degree of accuracy. For this, we used four different models and tuned their parameters to achieve the best results. The four models are:
  - Logistic Regression
  - Decision Trees
  - Support Vector Machine (SVM)
  - K-Nearest Neighbor algorithm (KNN)
- The first step is to read the data and standardize the feature values to ensure that they are all on the same scale. Without standardization, some feature values may end up skewing the model's performance and result in suboptimal parameter configuration. We use StandardScaler's fit\_transform() function for this step.
- The next step is to split our data into a training set and testing set. We use the train\_test\_split() function for this. We use a ratio of 80%-20% for the train-test split.
- Next, for each model, we set up the model's configuration parameters and use the GridSearchCV method to run the model using a combination of cross-validation and different values for the configuration parameters to achieve the best performance with the training data. At the end of this step, we are able to view the most optimal values of the configuration parameters and the resulting accuracy of the model.
- The next step is to evaluate the performance of the tuned model with the testing data to see if it continues to behave optimally. Finally, we plot a confusion matrix to observe the true positives vs the false positives. This tells us whether the model correctly predicts that the booster will successfully land or not for the testing data.
- Github URL:  
[https://github.com/swar00p/data\\_analytics/blob/bd5425ac086d0bcbec0ccb058e21bbab7e7a81a/Capstone/SpaceX-Machine-Learning-Prediction-Part-5-v1.ipynb](https://github.com/swar00p/data_analytics/blob/bd5425ac086d0bcbec0ccb058e21bbab7e7a81a/Capstone/SpaceX-Machine-Learning-Prediction-Part-5-v1.ipynb)

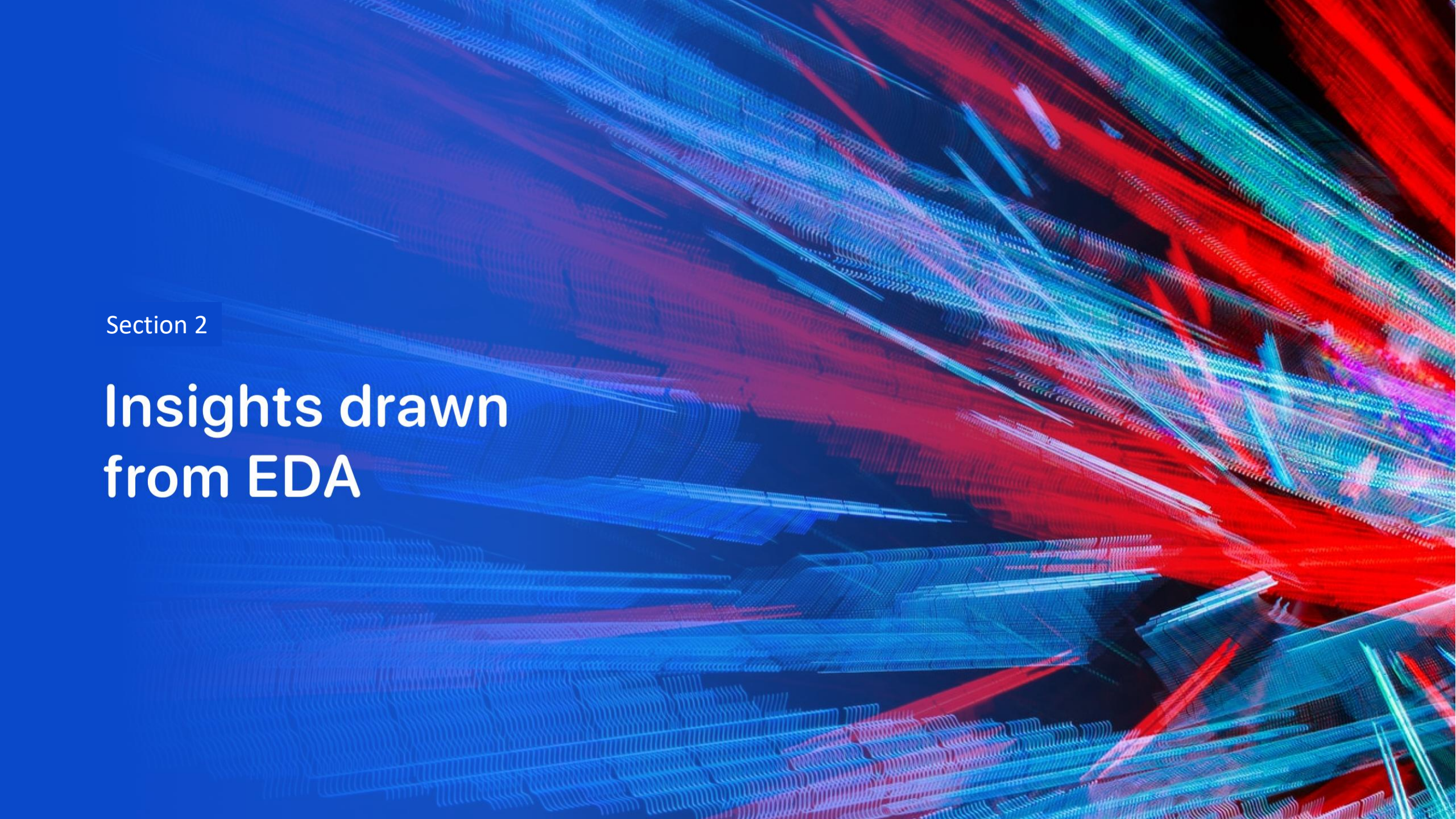


# Results

---

- Exploratory data analysis results
  - Higher payload masses seem to adversely impact average launch success rates
  - The orbits with the highest launch success rates are GEO, HEO, SSO & ES-L1
  - For heavier payloads, the orbit types LEO, PO & ISS seem to result in higher launch success rates
- Interactive analytics demo in screenshots
  - Most launch sites are located closer to the equator
  - Most launch sites appear to be near the coastline
  - Most launch sites appear to be built far from populated places like towns and cities
- Predictive analysis results
  - Three of the four models (Logistic Regression, SVM & KNN) produced similar levels of accuracy with the testing data. The one model that performed sub-optimally was the Decision Tree model.



The background of the slide is an abstract composition. It features a dark blue field on the left side, which transitions into a complex pattern of diagonal streaks and lines in shades of blue, red, and cyan on the right. These streaks have a textured, almost woven appearance, suggesting a digital or data-driven theme. The overall effect is dynamic and modern.

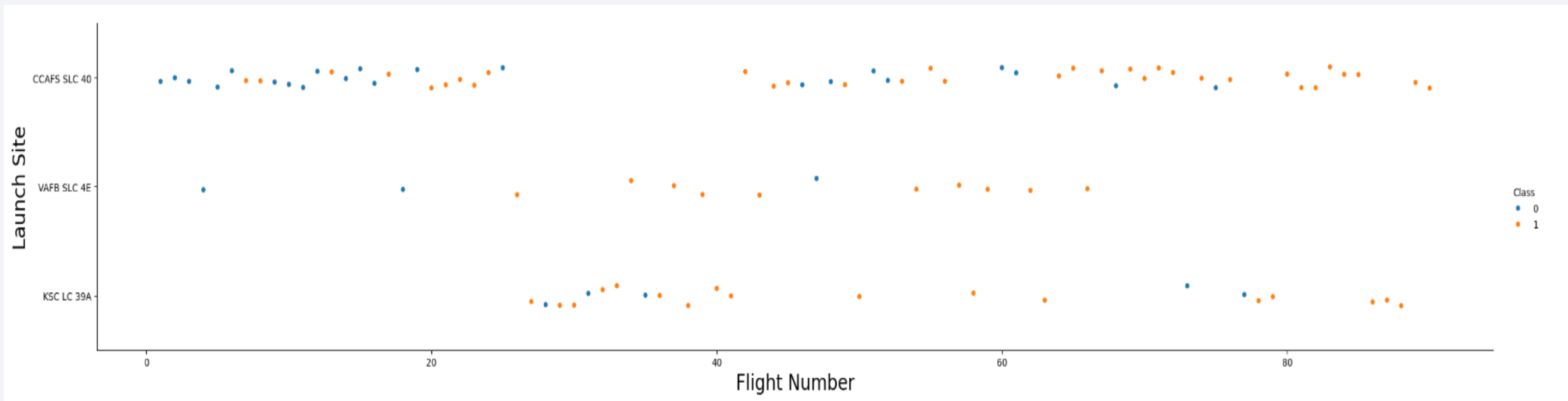
Section 2

# Insights drawn from EDA



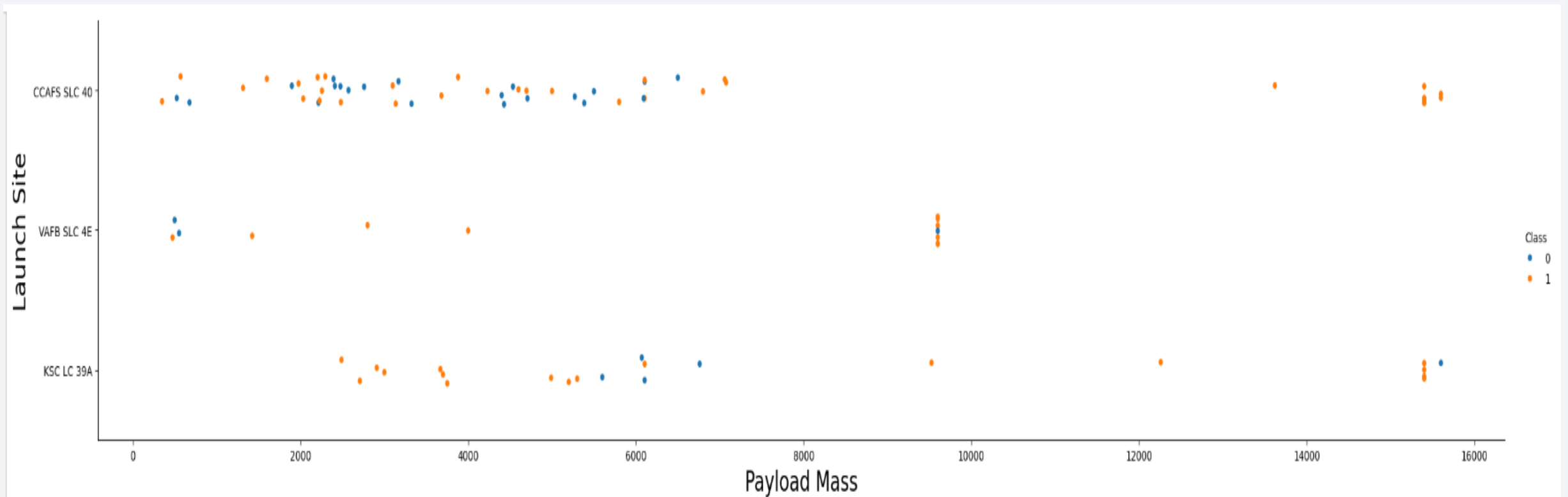
# Flight Number vs. Launch Site

- As the flight number increases (i.e. for later flights), it seems that one of the launch sites (VAFB SLC-4E) stops getting used. Also, a significantly higher number of launches happened from one of the launch sites (CCAFS SLC-40). Finally, the proportion of successful launches seems to increase with increasing flight numbers.



# Payload vs. Launch Site

- One of the launch sites has no launches with payload mass greater than 10000 kg. Also, there is generally a high concentration of payloads between 2000 kg and 7000 kg.

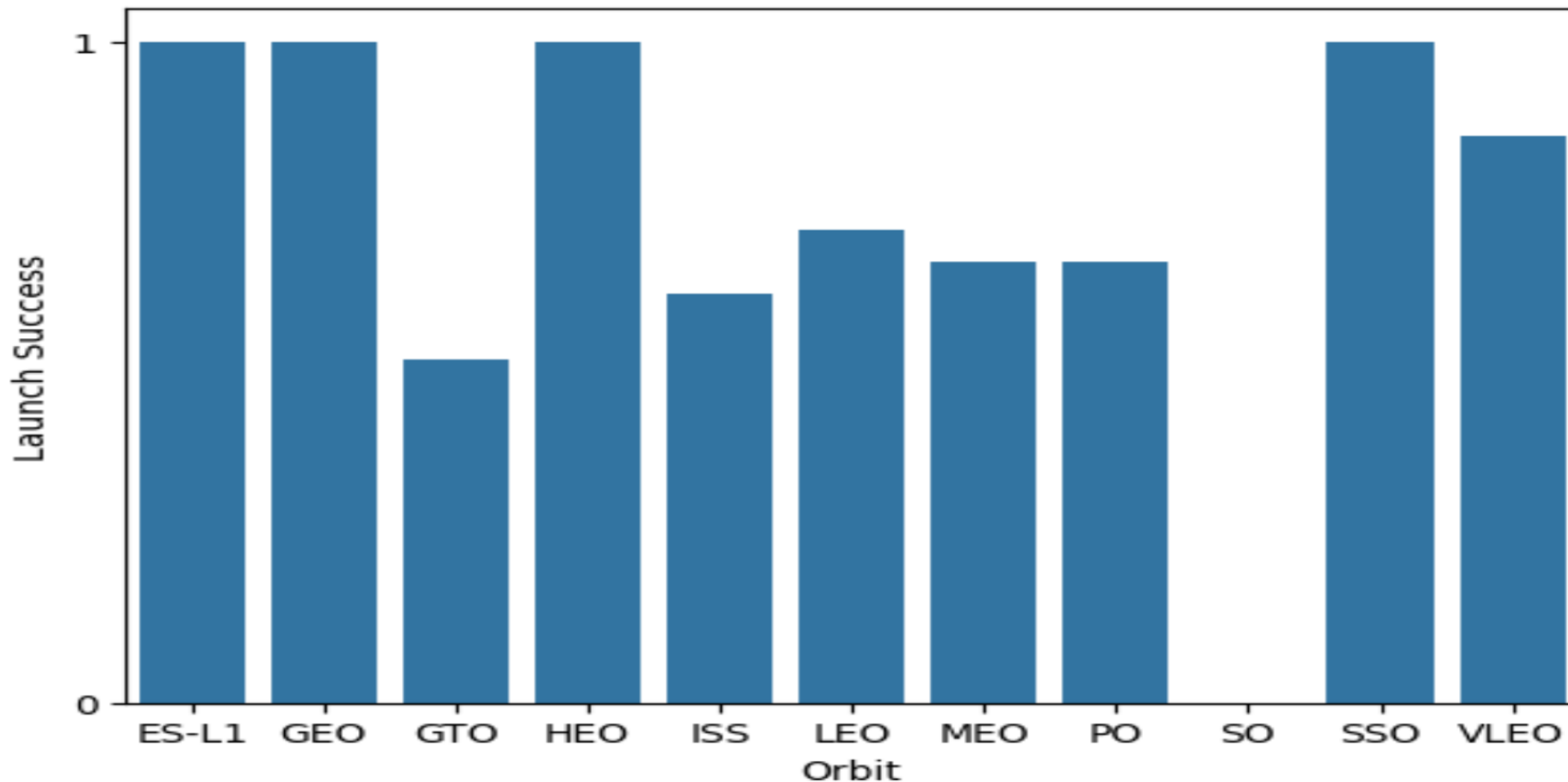




# Success Rate vs. Orbit Type

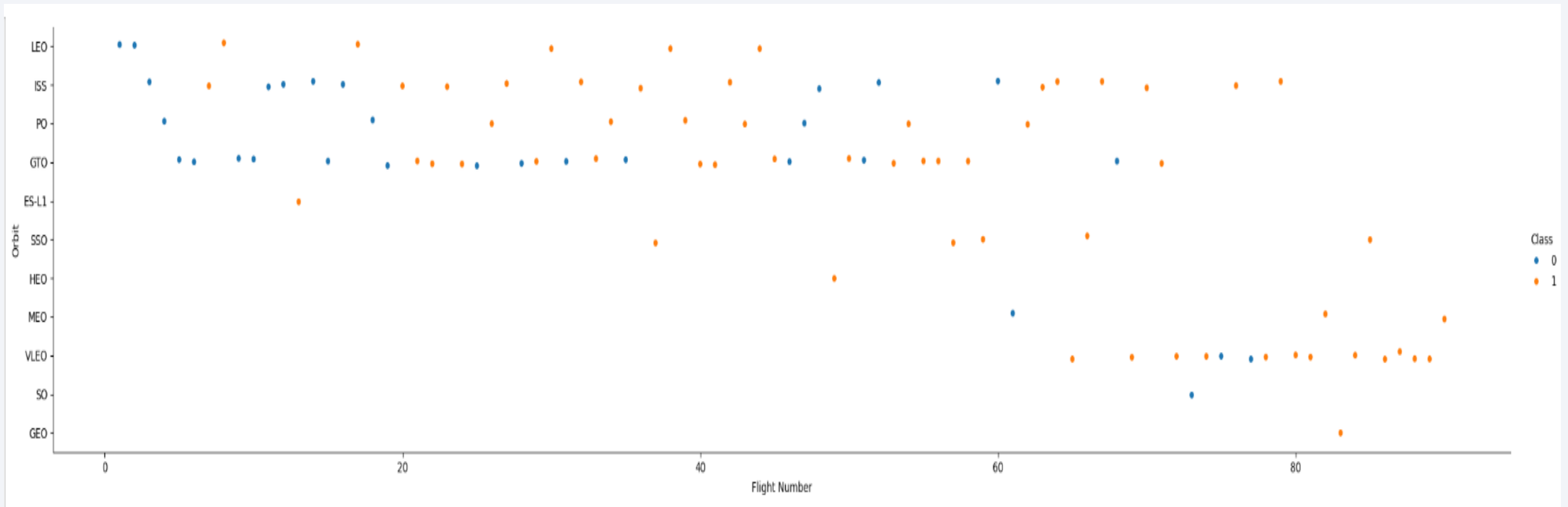
---

- The orbits with the highest success rate are ES-L1, GEO, HEO and SSO.



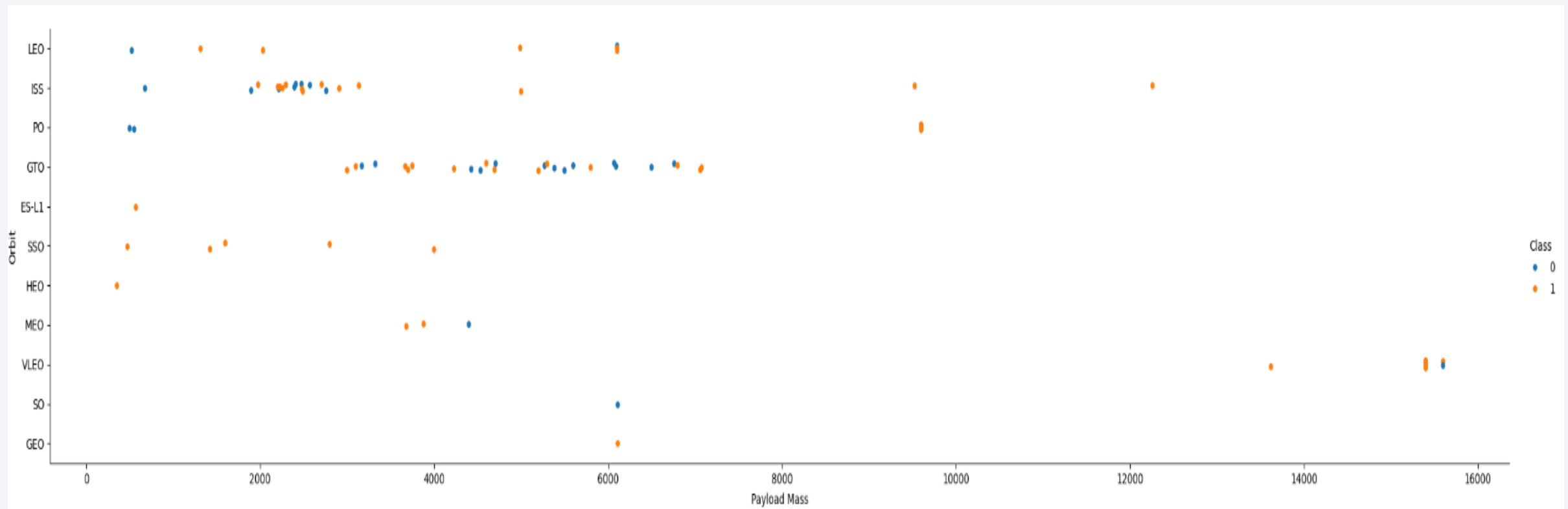
# Flight Number vs. Orbit Type

- Generally speaking, the orbits SEO, HEO, MEO, VLEO, SO and GEO appeared for later flight numbers. The earlier flight numbers were mostly for GTO, PO, LEO and ISS



# Payload vs. Orbit Type

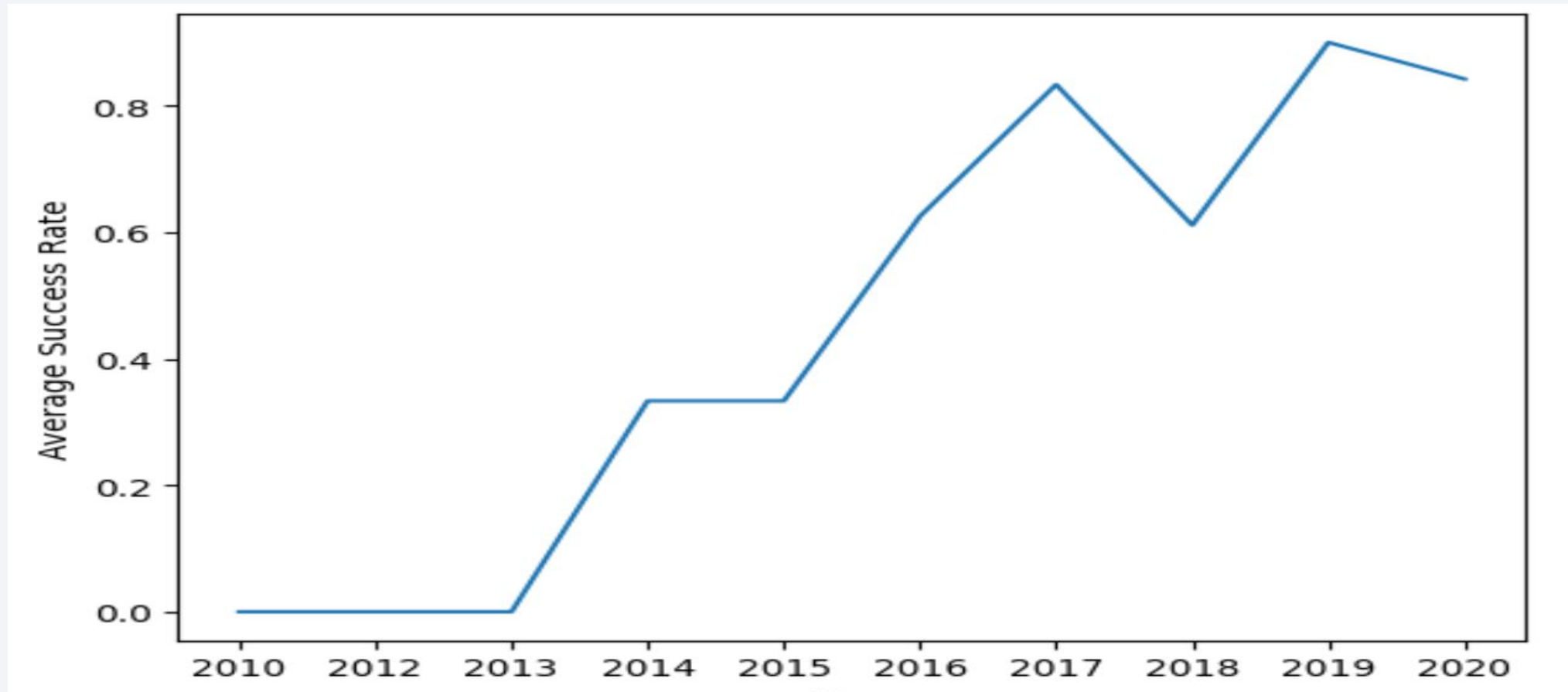
- For heavier payloads, it seems that there are more successful launches for LEO, ISS & PO orbit types.



# Launch Success Yearly Trend

---

- Generally speaking, a clear upward trend was observed in success rate over the years from 2009 to 2021, except for a slight decrease in 2018.



# All Launch Site Names

---

- Unique launch sites
  - `SELECT DISTINCT Launch_Site FROM SPACEXTABLE;`
  - The `DISTINCT` SQL keyword ensures that duplicate launch sites are not returned by the SQL query.

| Launch_Site  |
|--------------|
| CCAFS LC-40  |
| VAFB SLC-4E  |
| KSC LC-39A   |
| CCAFS SLC-40 |



# Launch Site Names Begin with 'CCA'

- `SELECT * FROM SPACEXTABLE WHERE Launch_Site LIKE 'CCA%' LIMIT 5;`
- The “LIKE ‘CCA%’” SQL clause fetches records that have launch site names beginning with ‘CCA’. The ‘LIMIT 5’ clauses ensures that a maximum of 5 records are returned by the query.

| Date       | Time (UTC) | Booster_Version | Launch_Site | Payload                                                       | PAYLOAD_MASS_KG_ | Orbit     | Customer        | Mission_Outcome | Lar |
|------------|------------|-----------------|-------------|---------------------------------------------------------------|------------------|-----------|-----------------|-----------------|-----|
| 2010-06-04 | 18:45:00   | F9 v1.0 B0003   | CCAFS LC-40 | Dragon Spacecraft Qualification Unit                          | 0                | LEO       | SpaceX          | Success         | Fai |
| 2010-12-08 | 15:43:00   | F9 v1.0 B0004   | CCAFS LC-40 | Dragon demo flight C1, two CubeSats, barrel of Brouere cheese | 0                | LEO (ISS) | NASA (COTS) NRO | Success         | Fai |
| 2012-05-22 | 7:44:00    | F9 v1.0 B0005   | CCAFS LC-40 | Dragon demo flight C2                                         | 525              | LEO (ISS) | NASA (COTS)     | Success         |     |
| 2012-10-08 | 0:35:00    | F9 v1.0 B0006   | CCAFS LC-40 | SpaceX CRS-1                                                  | 500              | LEO (ISS) | NASA (CRS)      | Success         |     |
| 2013-03-01 | 15:10:00   | F9 v1.0 B0007   | CCAFS LC-40 | SpaceX CRS-2                                                  | 677              | LEO (ISS) | NASA (CRS)      | Success         |     |

# Total Payload Mass

---

- `SELECT SUM(PAYLOAD_MASS__KG_) FROM SPACEXTABLE WHERE Customer LIKE 'NASA (CRS)';`
- The `SUM()` SQL function adds the `PAYLOAD_MASS__KG_` column values for all selected rows.

| <b>SUM(PAYLOAD_MASS__KG_)</b> |
|-------------------------------|
| 45596                         |

# Average Payload Mass by F9 v1.1

---

- `SELECT AVG(PAYLOAD_MASS__KG_) FROM SPACEXTABLE WHERE Booster_Version LIKE 'F9 v1.1'`
- The `AVG()` SQL function computes the average of the `PAYLOAD_MASS__KG_` column for all the selected rows.

| <b>AVG(PAYLOAD_MASS__KG_)</b> |
|-------------------------------|
| 2928.4                        |

# First Successful Ground Landing Date

---

- `SELECT MIN(Date) FROM SPACEXTABLE WHERE Landing_Outcome = 'Success (ground pad)';`
- The `MIN()` SQL function returns the minimum value of the `Date` column for all the rows that were selected by the `WHERE` condition. In our case, the minimum value of the `Date` column happens to be the earliest date.

| <b>MIN(Date)</b> |
|------------------|
| 2015-12-22       |

## Successful Drone Ship Landing with Payload between 4000 and 6000

---

- `SELECT Booster_Version FROM SPACEXTABLE WHERE Landing_Outcome = 'Success (drone ship)' AND PAYLOAD_MASS__KG_ BETWEEN 4000 AND 6000;`
- In this query, we use two conditions to retrieve the records we are interested in. The first is to select successful landings on a drone ship and the other to select payloads in a certain weight range.

| Booster_Version |
|-----------------|
| F9 FT B1022     |
| F9 FT B1026     |
| F9 FT B1021.2   |
| F9 FT B1031.2   |

# Total Number of Successful and Failure Mission Outcomes

---

- `SELECT SUBSTR(Landing_Outcome, 1, 7) AS Landing_Outcome, COUNT(*) FROM SPACEXTABLE WHERE Landing_Outcome LIKE 'Success%' OR Landing_Outcome LIKE 'Failure%' GROUP BY SUBSTR(Landing_Outcome, 1, 7);`
- This is a complex query that takes advantage of the fact that both 'Success' and 'Failure' have 7 letters each.

| Landing_Outcome | COUNT(*) |
|-----------------|----------|
| Failure         | 10       |
| Success         | 61       |



# Boosters Carried Maximum Payload

- `SELECT Booster_Version FROM SPACEXTABLE WHERE PAYLOAD_MASS__KG_ = (SELECT MAX(PAYLOAD_MASS__KG_) FROM SPACEXTABLE);`
- We need a sub-query in order to first get the maximum value of the payload mass column and then use that to get the booster versions that correspond to that payload mass.

| Booster_Version |
|-----------------|
| F9 B5 B1048.4   |
| F9 B5 B1049.4   |
| F9 B5 B1051.3   |
| F9 B5 B1056.4   |
| F9 B5 B1048.5   |
| F9 B5 B1051.4   |
| F9 B5 B1049.5   |
| F9 B5 B1060.2   |
| F9 B5 B1058.3   |
| F9 B5 B1051.6   |

# 2015 Launch Records

---

- `SELECT SUBSTR(Date, 6, 2) AS MONTH, Landing_Outcome, Booster_Version, Launch_Site FROM SPACEXTABLE WHERE Landing_Outcome = 'Failure (drone ship)' and SUBSTR(Date, 0, 5) = '2015';`
- Since SQLite3 does not support month names, we have to use the SUBSTR() function to get the month number instead

| <b>MONTH</b> | <b>Landing_Outcome</b> | <b>Booster_Version</b> | <b>Launch_Site</b> |
|--------------|------------------------|------------------------|--------------------|
| 01           | Failure (drone ship)   | F9 v1.1 B1012          | CCAFS LC-40        |
| 04           | Failure (drone ship)   | F9 v1.1 B1015          | CCAFS LC-40        |

## Rank Landing Outcomes Between 2010-06-04 and 2017-03-20

- `SELECT Landing_Outcome, COUNT(*) AS NumLandings FROM SPACEXTABLE WHERE Date BETWEEN '2010-06-04' AND '2017-03-20' GROUP BY Landing_Outcome ORDER BY NumLandings DESC;`
- We have to use a `GROUP BY` clause in order to get a count of each landing outcome and then we have to use the `ORDER BY...DESC` clause to sort the results in descending order

| Landing_Outcome        | NumLandings |
|------------------------|-------------|
| No attempt             | 10          |
| Success (drone ship)   | 5           |
| Failure (drone ship)   | 5           |
| Success (ground pad)   | 3           |
| Controlled (ocean)     | 3           |
| Uncontrolled (ocean)   | 2           |
| Failure (parachute)    | 2           |
| Precluded (drone ship) | 1           |

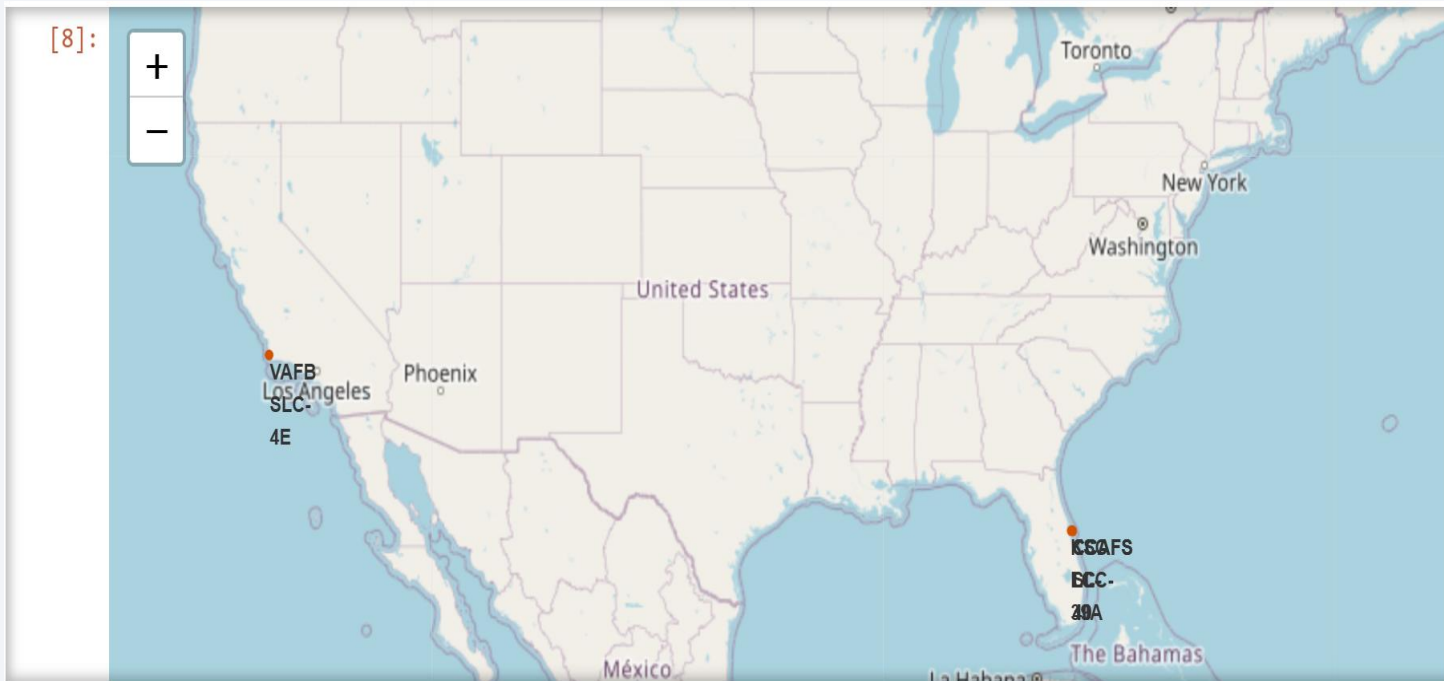
A satellite view of Earth from space, showing the curvature of the planet and city lights at night. The background is a deep blue gradient.

Section 3

# Launch Sites Proximities Analysis

# Display all launch sites

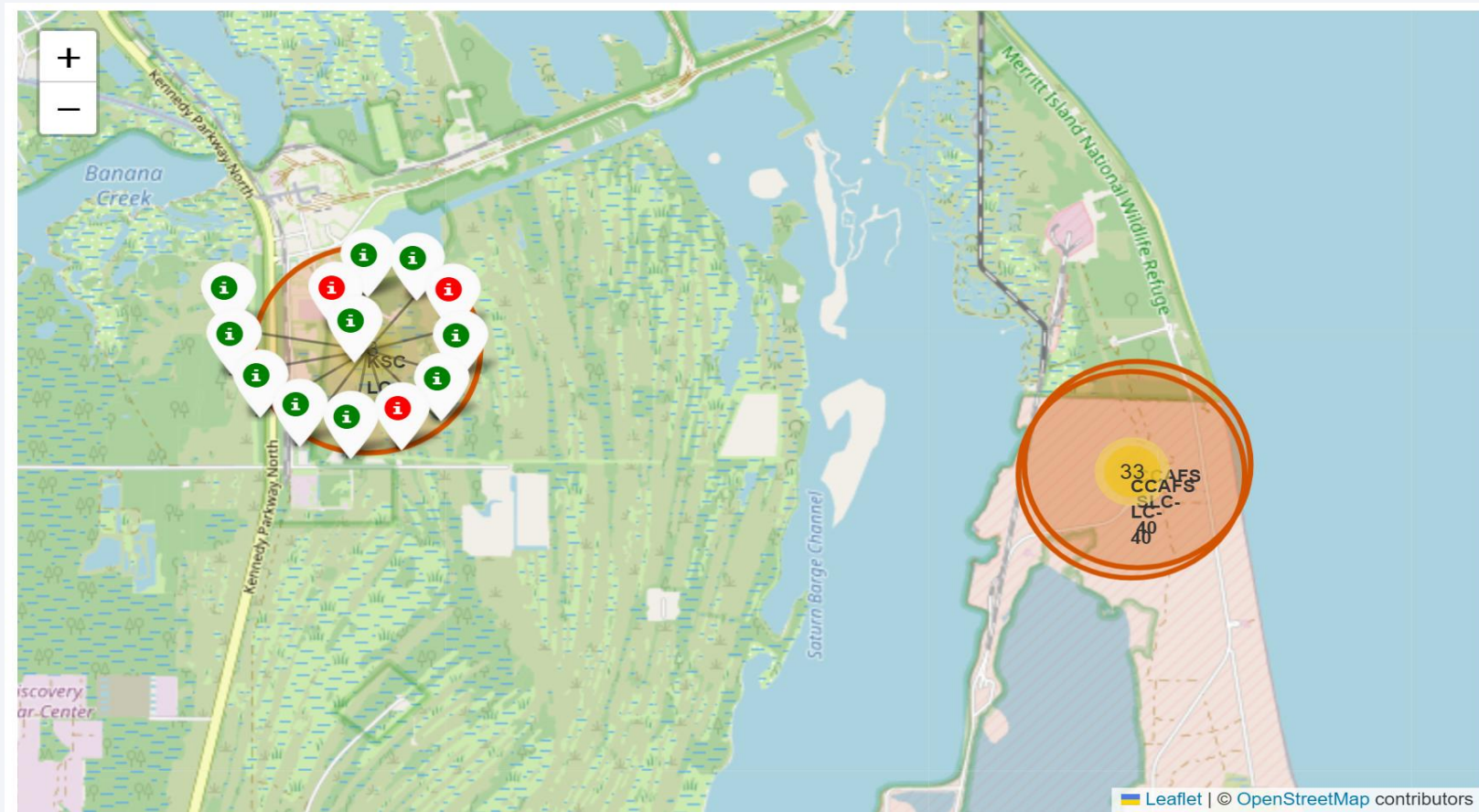
- From the screenshot, we can observe that all launch sites are located in the southern part of the US, closer to the equator.
- Further, it may be observed that all launch sites are very close to the coastline





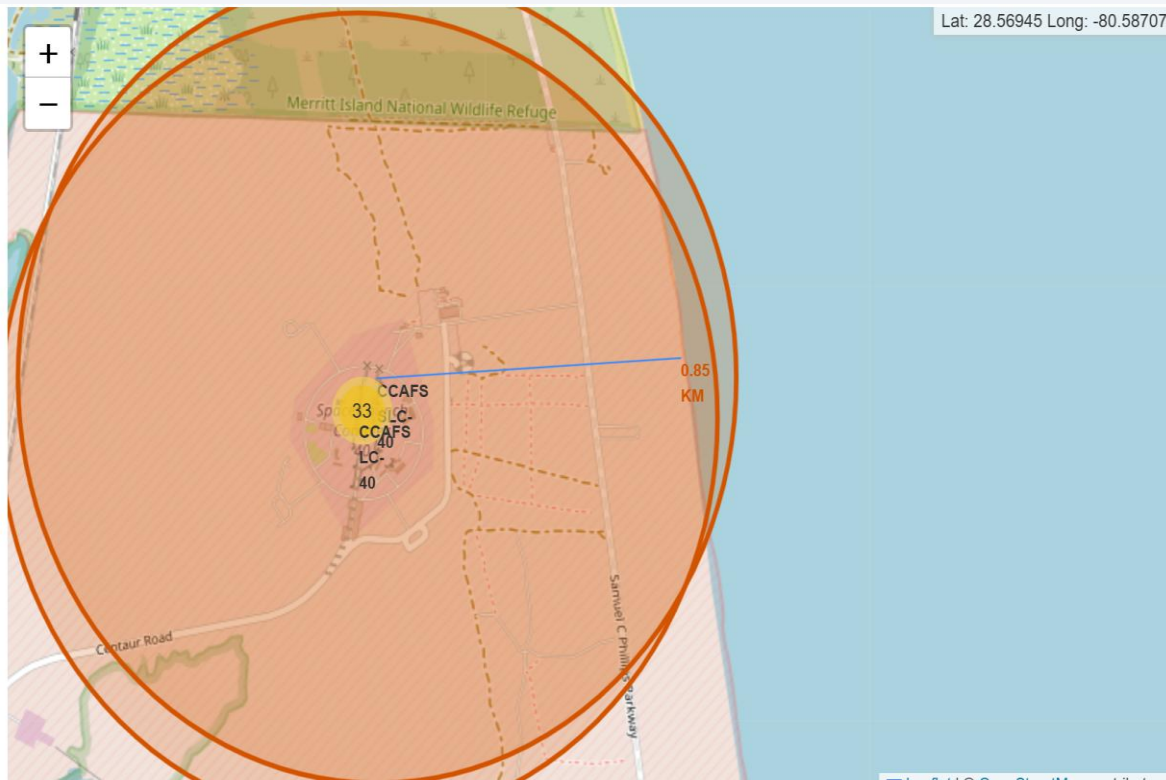
# Mark the success/failed launches for each site on the map

- Display all launches in each launch site along with colors to indicate success or failure. Clicking on a launch site automatically zooms in on it and displays all launches.

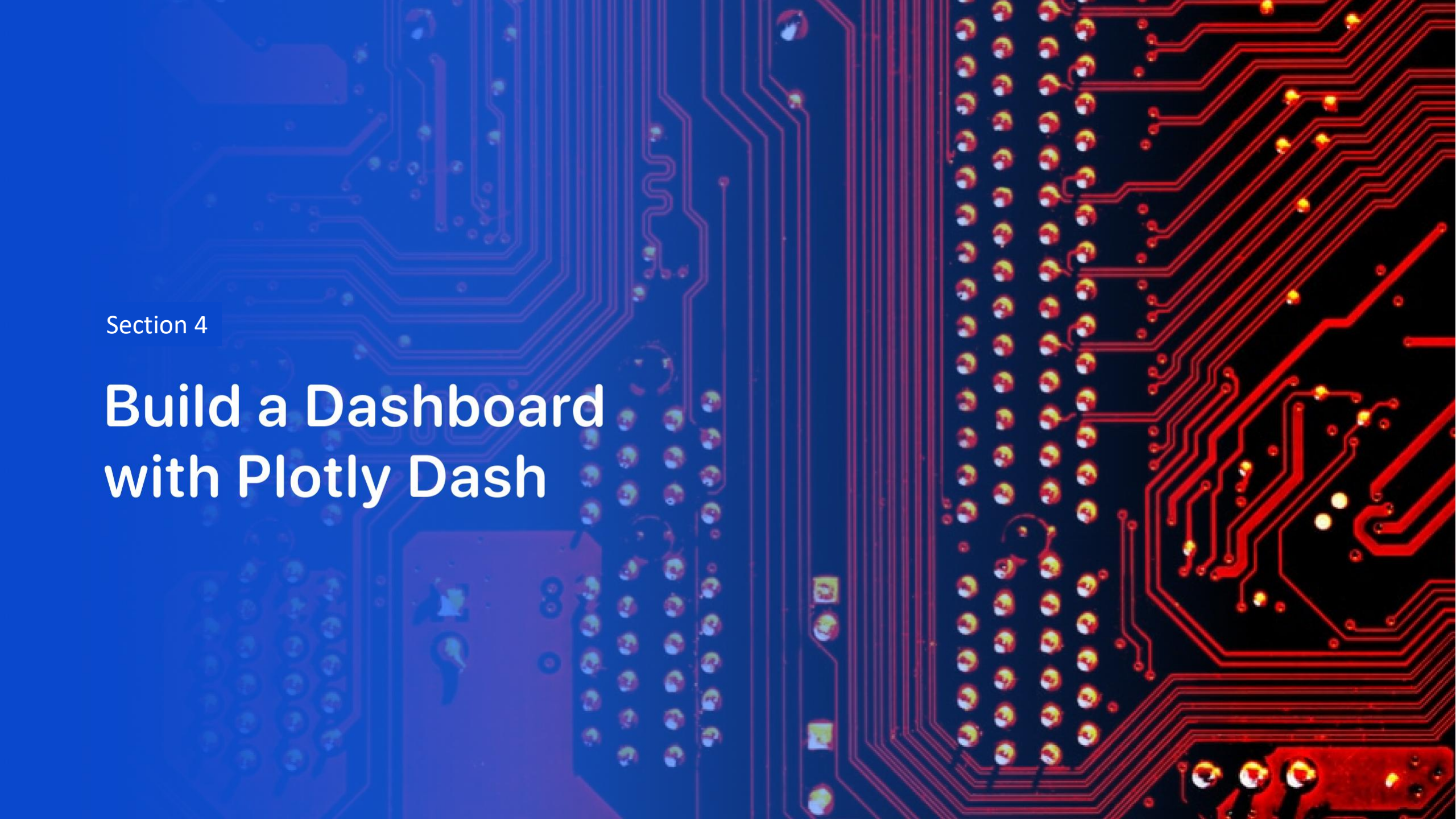


# Calculate the distances between a launch site to its proximities

- The first screenshot shows the distance between CCAFS-SLC40 and the coastline.
- The second screenshot shows the proximity of the VAFB SLC-4E to the nearest city (Lompoc).





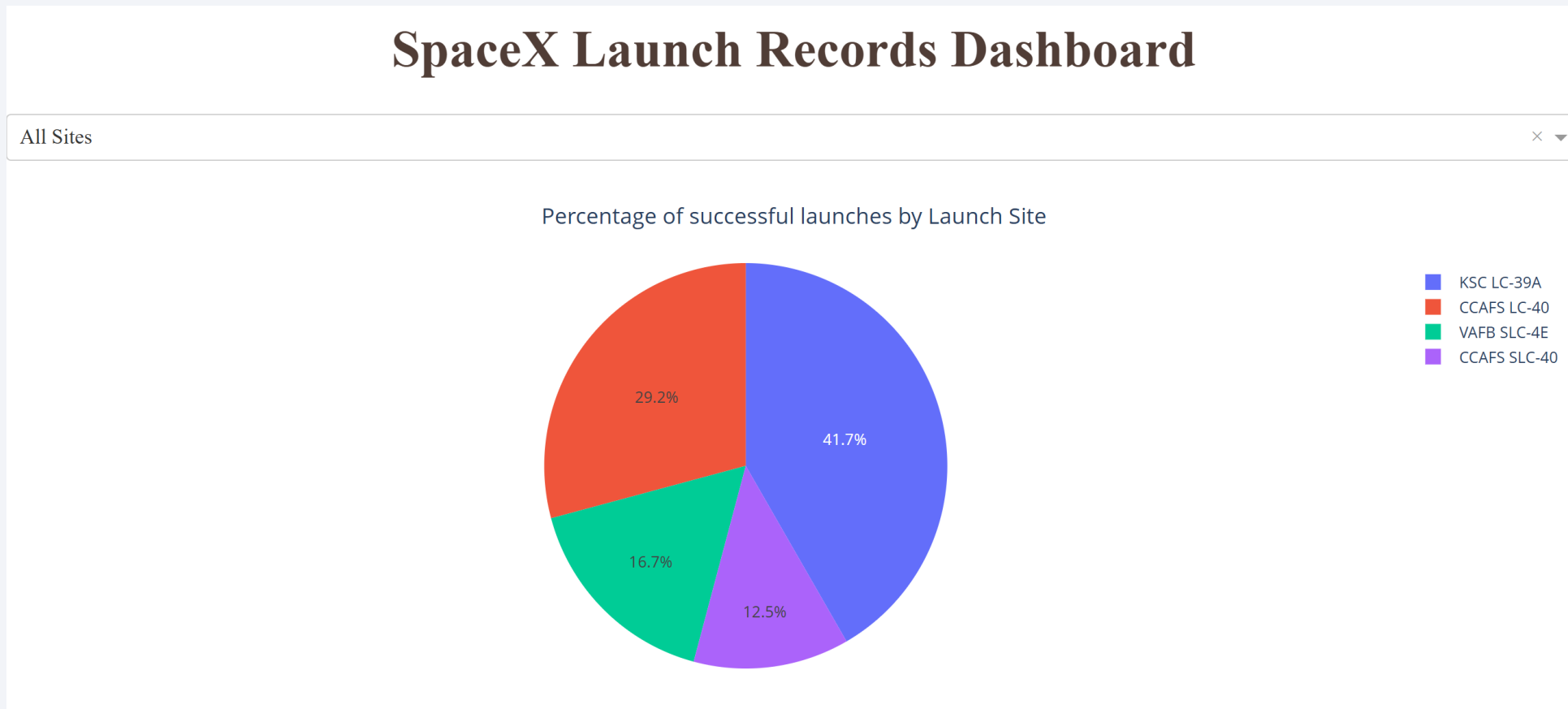


Section 4

# Build a Dashboard with Plotly Dash

# Percentage of successful launches by Launch Site

- KSC LC-39A features the highest percentage of successful launches among all sites, followed by CCAFS LC-40.



# Launch outcome distribution for KSC LC-39A

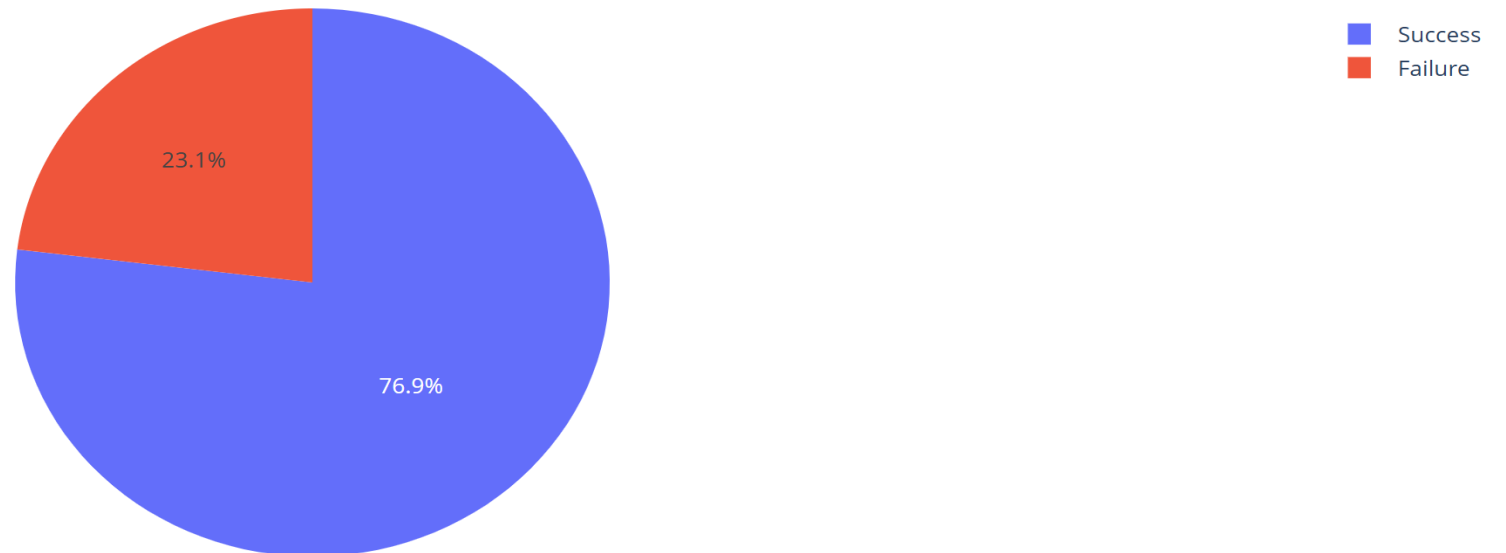
- Distribution of launch outcomes for KSC LC-39A which constitutes the majority of the successful launches across all launch sites.

## SpaceX Launch Records Dashboard

KSC LC-39A

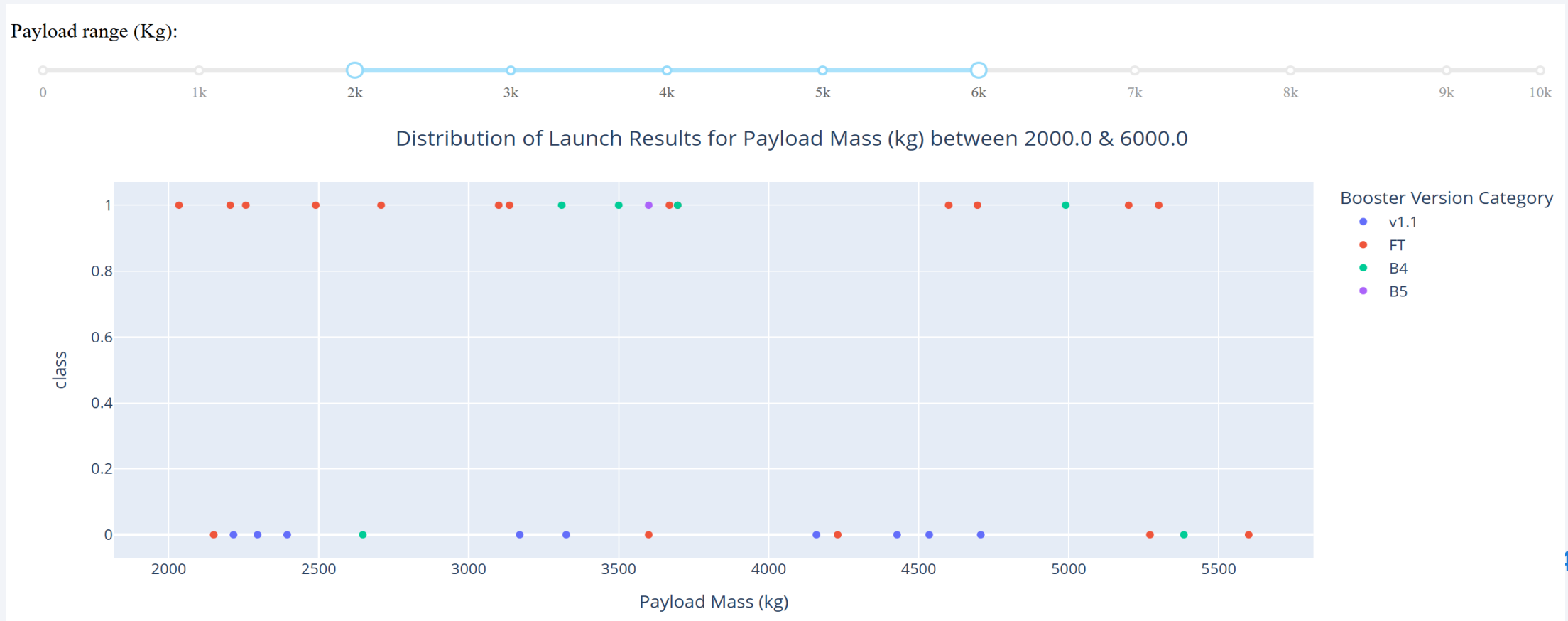


Distribution of Launch Results for site KSC LC-39A



# Launch outcomes based on payload

- Distribution of launch outcomes across all launch sites for payloads between 2000 kg and 6000 kg.





Section 5

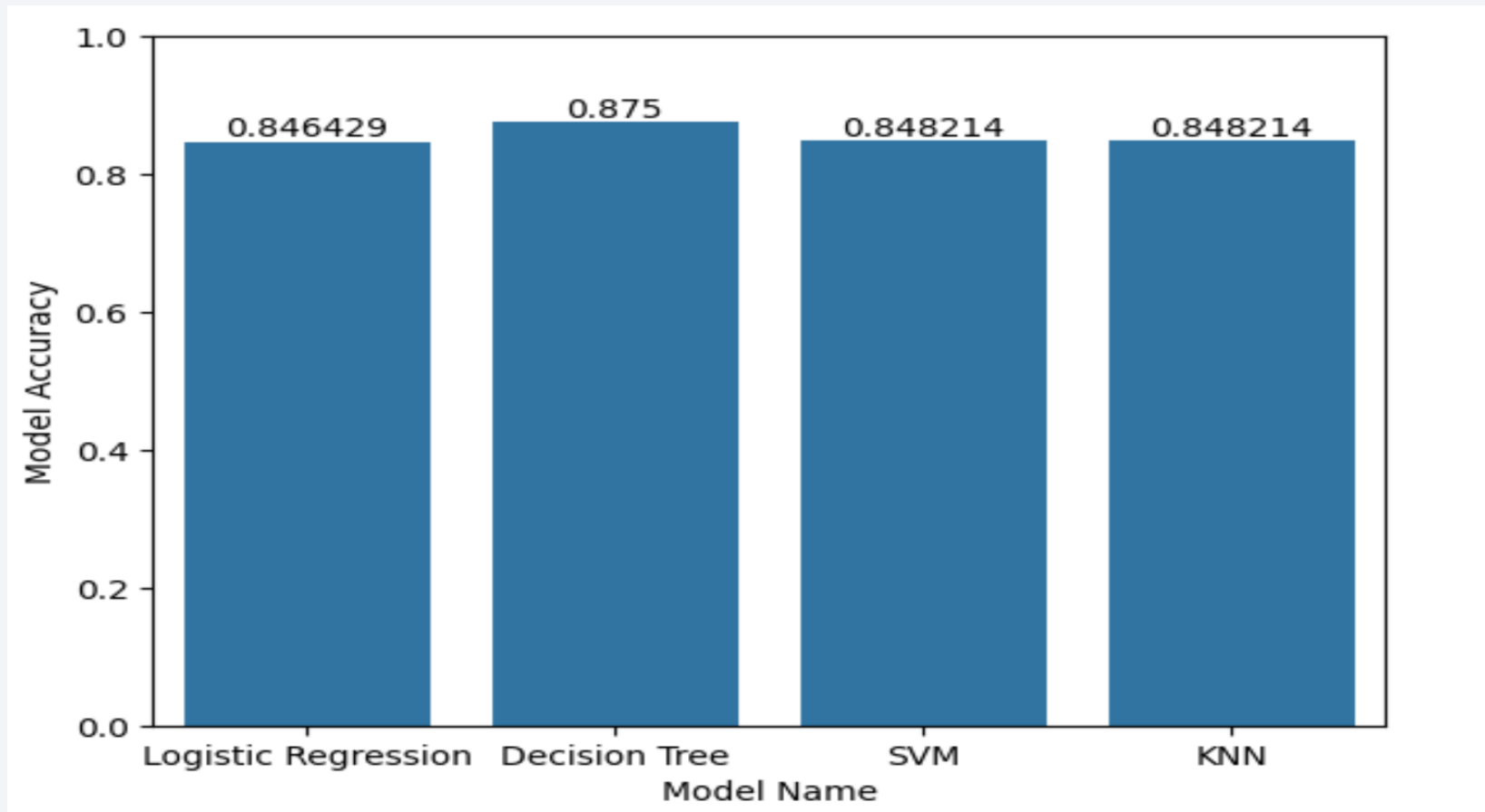
# Predictive Analysis (Classification)



# Classification Accuracy

---

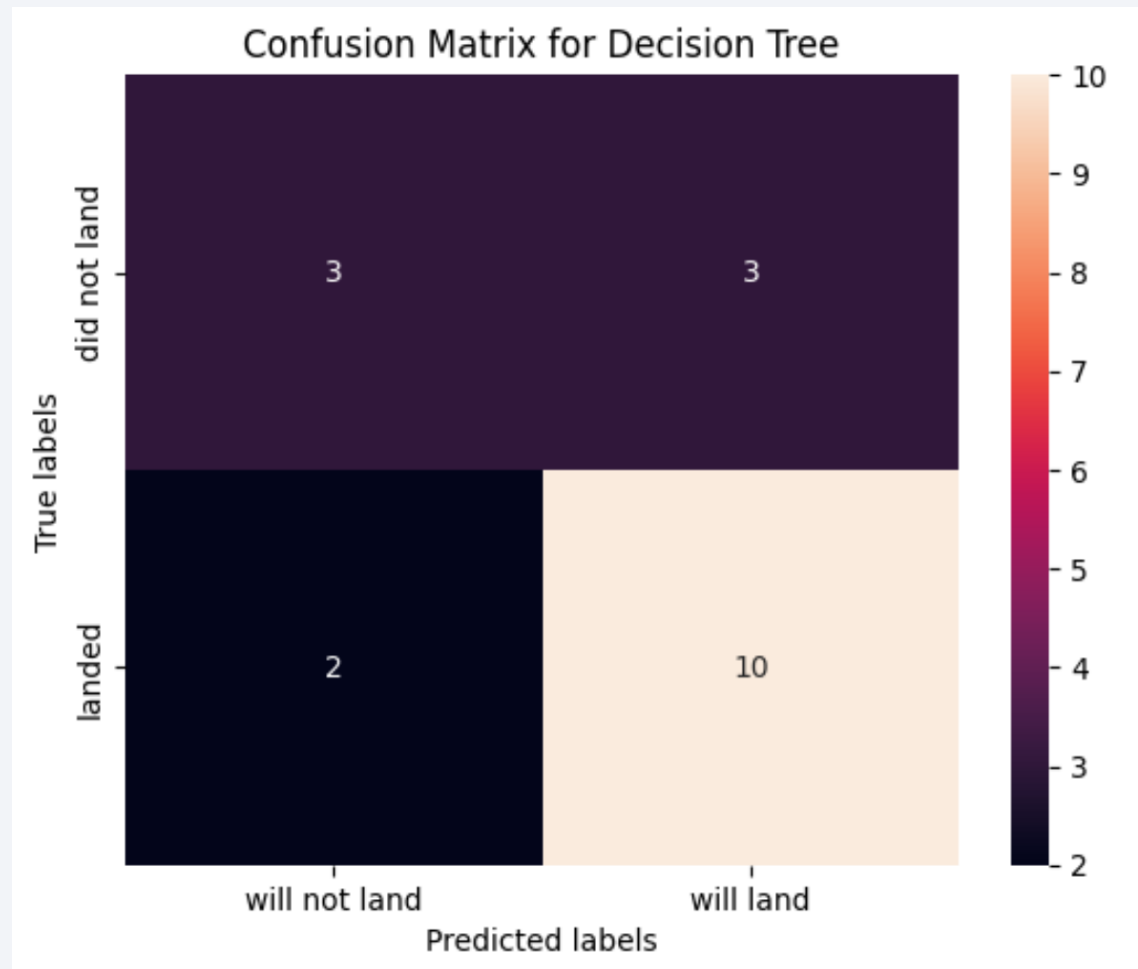
- A bar chart visualizing the accuracy of each model is shown below.
- As observed, the Decision Tree model seems to be slightly more accurate than the other models.





# Confusion Matrix

- The Decision Tree model was the best performer based on its accuracy score. It predicted 10 True Positive and 3 False Positive outcomes.



# Conclusions

---

- Based on the analysis and modeling that was done during this project, we can conclude that payload mass and orbit have a fairly large impact on the launch outcome.
- There are certain key criteria that are shared by all launch sites like proximity to coastline, minimum distance from populated zones, etc.

# Appendix

---

Thank you!

